

Worksheet No. - 05

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 2nd

Subject Name: Artificial Intelligence Tools

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 02.04.2026

Subject Code: 25CAP-657

Aim/Overview of the practical: Implementing hill climbing.

Objective: The objective of implementing the Hill Climbing Algorithm is to understand how local search techniques work in finding optimal or near-optimal solutions. It helps in learning how to iteratively move from one state to a better neighboring state based on a given evaluation function, and to analyze challenges like local maxima and plateaus in optimization problems.

Procedure/Algorithm/Code

Algorithm

1. Start with an initial solution (current state).
2. Evaluate its value using a fitness function.
3. Generate all possible neighboring states.
4. Choose the neighbor with the best value.
5. If the neighbor is better than the current state, move to it.
6. Repeat steps 3–5 until no better neighbor is found.
7. Return the current state as the final solution.

Code

```
import random

import numpy as np

import matplotlib.pyplot as plt

class Graph:

    def __init__(self):
```

```
self.adj_list = {}

def add_edge(self, node, neighbor):

    if node not in self.adj_list:

        self.adj_list[node] = []

    self.adj_list[node].append(neighbor)

def get_neighbors(self, node):

    return self.adj_list.get(node, [])

def objective_function(x):

    return -x**2 + 4*x

def heuristic(x):

    return 4 - 2*x

def hill_climb(start_x, step_size=0.1, max_iterations=1000):

    current_x = start_x

    current_value = objective_function(current_x)

    x_history = [current_x]

    y_history = [current_value]

    heuristic_values = [heuristic(current_x)]

    graph = Graph()

    for _ in range(max_iterations):

        neighbors = [current_x + step_size, current_x - step_size]

        best_neighbor = current_x

        best_value = current_value

        for next_x in neighbors:

            next_value = objective_function(next_x)

            graph.add_edge(current_x, next_x)
```

```
if next_value > best_value:
```

```
    best_neighbor, best_value = next_x, next_value
```

```
if best_value > current_value:
```

```
    current_x, current_value = best_neighbor, best_value
```

```
    x_history.append(current_x)
```

```
    y_history.append(current_value)
```

```
    heuristic_values.append(heuristic(current_x))
```

```
else:
```

```
    break
```

```
return current_x, current_value, x_history, y_history, heuristic_values, graph
```

```
# RUN HILL CLIMBING
```

```
start_x = random.uniform(-7, 4)
```

```
best_x, best_value, x_hist, y_hist, h_values, graph = hill_climb(start_x)
```

```
# Plotting the function
```

```
x_vals = np.linspace(-2, 4, 100)
```

```
y_vals = objective_function(x_vals)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(x_vals, y_vals, label='Objective Function')
```

```
plt.scatter(x_hist, y_hist, color='red', label='Path Taken', zorder=3)
```

```
plt.scatter(best_x, best_value, color='green', marker='X', s=100, label='Best Solution', zorder=4)
```

```
plt.xlabel('x')
```

```
plt.ylabel('Objective Value')
```

```
plt.title('Hill Climbing Optimization')
```

```
plt.legend()
```

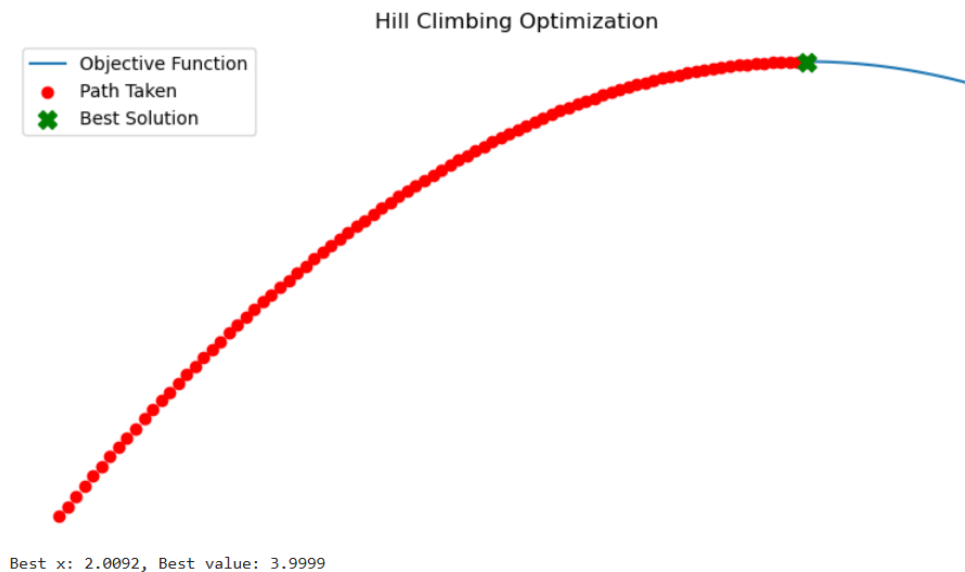
```
plt.grid()
```

```
plt.axis("off")
```

```
plt.show()
```

```
print(f'Best x: {best_x:.4f}, Best value: {best_value:.4f}')
```

Output



Learning outcomes (What I have learnt):

1. Understood the concept of the Hill Climbing Algorithm.
2. Learnt how local search and optimization techniques work.
3. Developed problem-solving skills using iterative improvement.